PRACTICAL- 8

MySQL Schema Design, ER Model, PDO Connectivity, and Prepared Statements

AIM / Problem Definition: Design the database for StudentHub with students, events and registrations. Create PHP database connection using PDO prepared statements.

Tools / Technology Used: MySQL, phpMyAdmin, PHP 8.x, PDO, Apache / XAMPP, VS Code

1. Project Implementation & Database Architecture

1.1 Project Overview (StudentHub Portal)

In Practicals 1 through 7, we developed the front-end user experience, interactive forms, dynamic catalog views, and server-side PHP processing for the StudentHub university portal. In this practical, we transition to the persistent data layer by designing and deploying a normalized MySQL relational database schema, establishing a secure PDO-based connectivity module, and validating the schema through phpMyAdmin seeded data and structural inspection.

1.2 Schema Design & Entity-Relationship Architecture

To support student life and campus engagement, the StudentHub database is built upon three primary relational entities and two auxiliary tracking tables:

• **Students Entity (students):** Stores comprehensive student credentials, academic profiles, authentication secrets (bcrypt-hashed passwords), and metadata (timestamps, role, profile image, bio).

• **Events Entity (events):** Manages university workshops, technical symposiums, hackathons, and cultural fests with rich descriptions, categories, badges, physical locations, date ranges, and capacity limits.

• **Registrations Entity (registrations):** Serves as a relational junction entity establishing an authenticated Many-to-Many relationship between students and events, carrying unique ticket codes, status, and timestamps.

• **Contact & Audit Logging (contact_inquiries, audit_logs):** Captures campus support tickets and maintains administrative audit logs for tracking system-level actions and data changes.

1.3 Secure Database Connectivity (db.php) & PDO Prepared Statements

Database connectivity is implemented in a centralized db.php configuration file using PHP Data Objects (PDO). PDO provides a unified, driver-agnostic interface for MySQL access with built-in support for prepared statements and parameterized queries:

• **Exception Handling Mode:** Configured with PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION to guarantee that database anomalies surface as catchable PHP exceptions rather than silent failures.

• **Native Prepared Statements:** PDO::ATTR_EMULATE_PREPARES => false forces MySQL's native prepared statement engine, providing true server-side query compilation and parameterized binding.

• **Character Encoding:** The DSN explicitly specifies charset=utf8mb4 to support full Unicode including emoji and international scripts across all database operations.

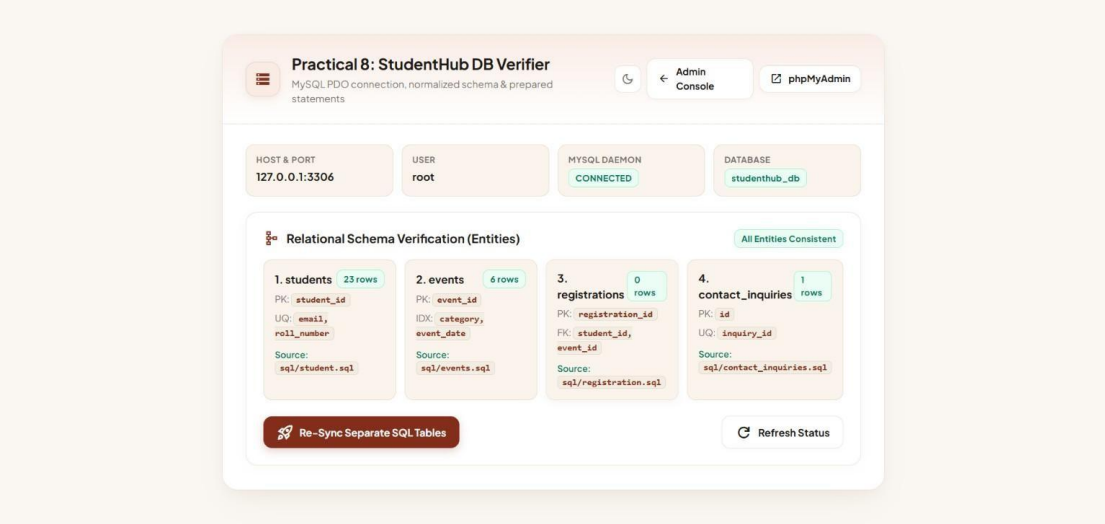
• **Fetch Mode:** PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC ensures all query results return associative arrays for clean key-based data access in PHP.

2. Output Screenshots & Database Verification

Below are the verified output screens from phpMyAdmin confirming successful schema deployment, table structures, seed data, and relational constraints:

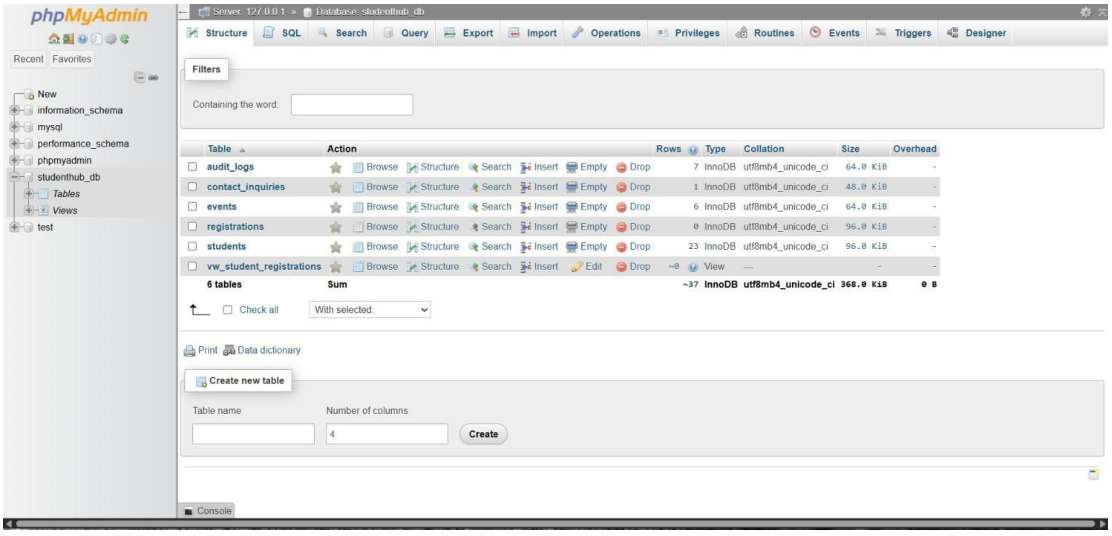
2.1 Output 1: PDO Database Connection Test (StudentHub DB Verifier)

The custom StudentHub DB Verifier dashboard verifies real-time connectivity status, displaying connected host (localhost), database name (studenthub_db), character set (utf8mb4), total table count, and student record count.



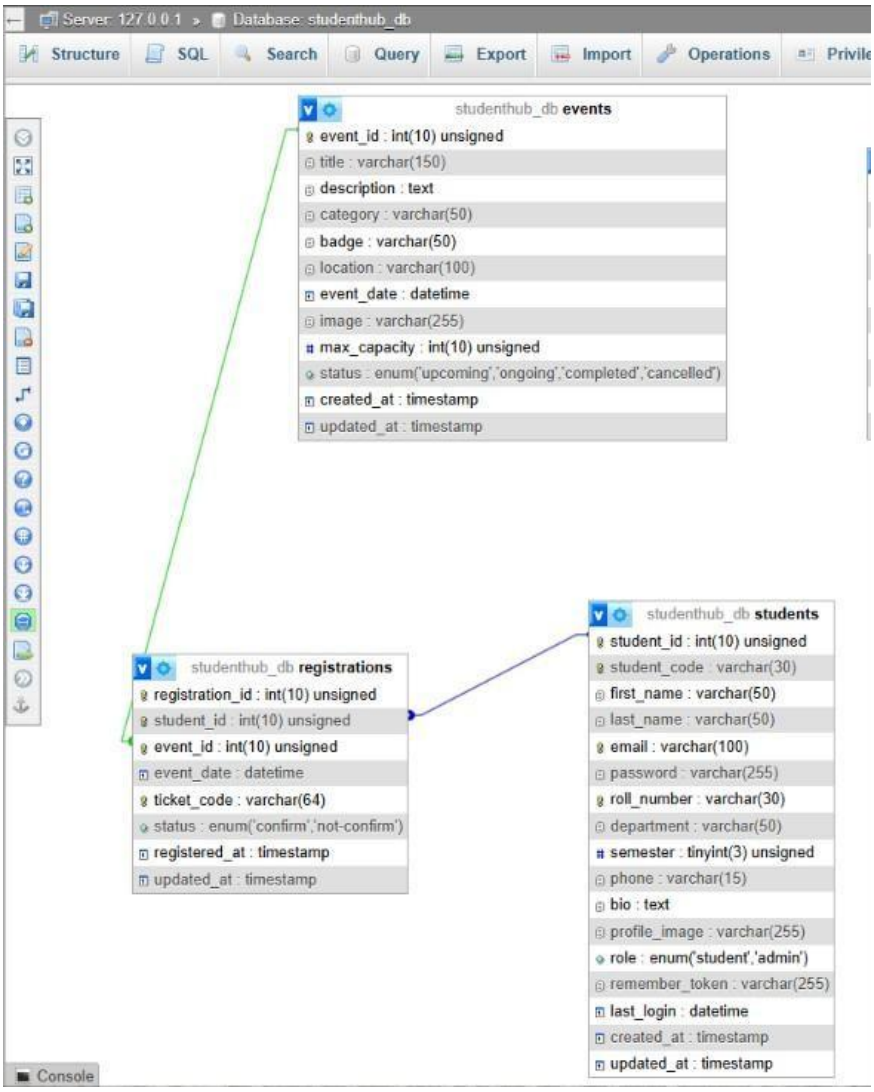
2.2 Output 2: Database Overview — studenthub_db Table Listing

phpMyAdmin interface displaying the overall structure of studenthub_db. It lists all database tables (audit_logs, contact_inquiries, events, registrations, students) with their respective storage engines (InnoDB), row counts, collation (utf8mb4_general_ci), and disk usage.



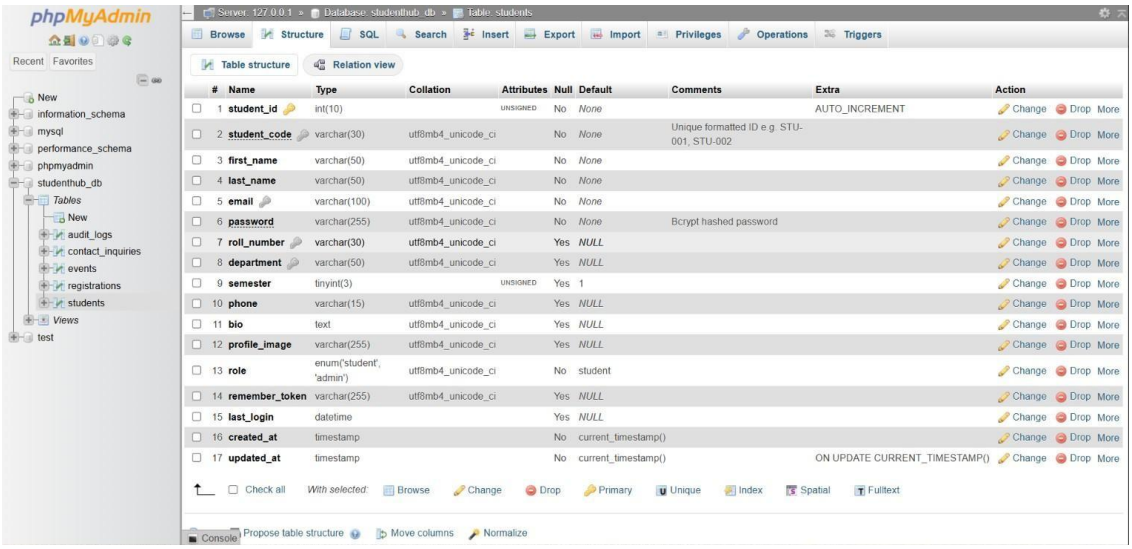
2.3 Output 3: Entity-Relationship Diagram (phpMyAdmin Designer)

Visual ER model rendered in phpMyAdmin Designer illustrating the relational topology: students (1:N) registrations (N:1) events, demonstrating the decomposed Many-to-Many relationship through the registrations junction table with foreign key lines.



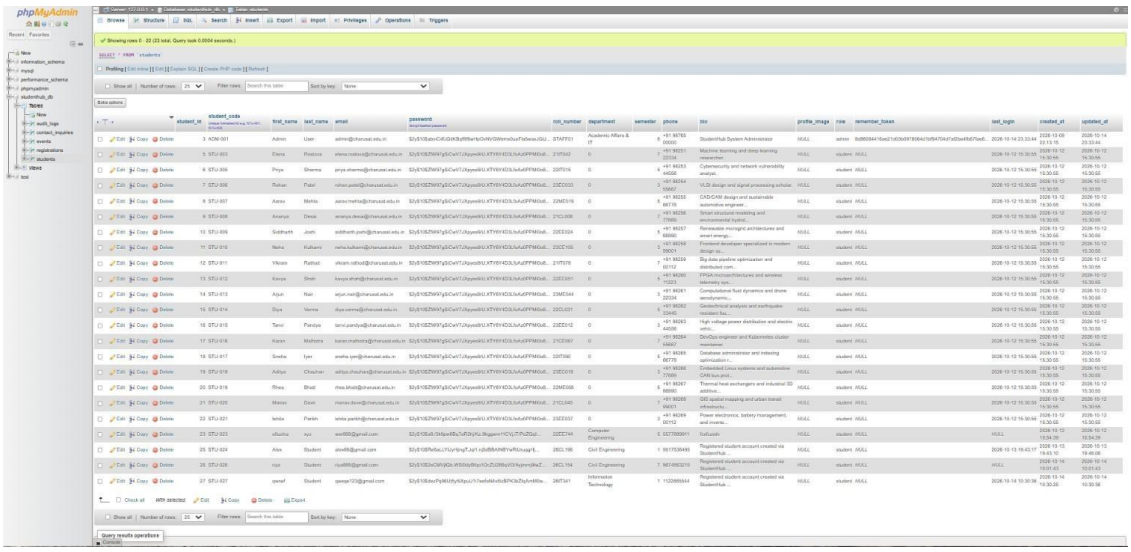
2.4 Output 4: Students Table Structure & Constraints

Complete schema specification for the students table featuring auto-incrementing primary key student_id, formatted student_code (STU-XXX), first_name, last_name, email (UNIQUE), bcrypt password column (VARCHAR 255), roll_number (UNIQUE), department, semester, phone, bio, profile_image, role ENUM (student/admin), remember_token, last_login, and timestamp fields.



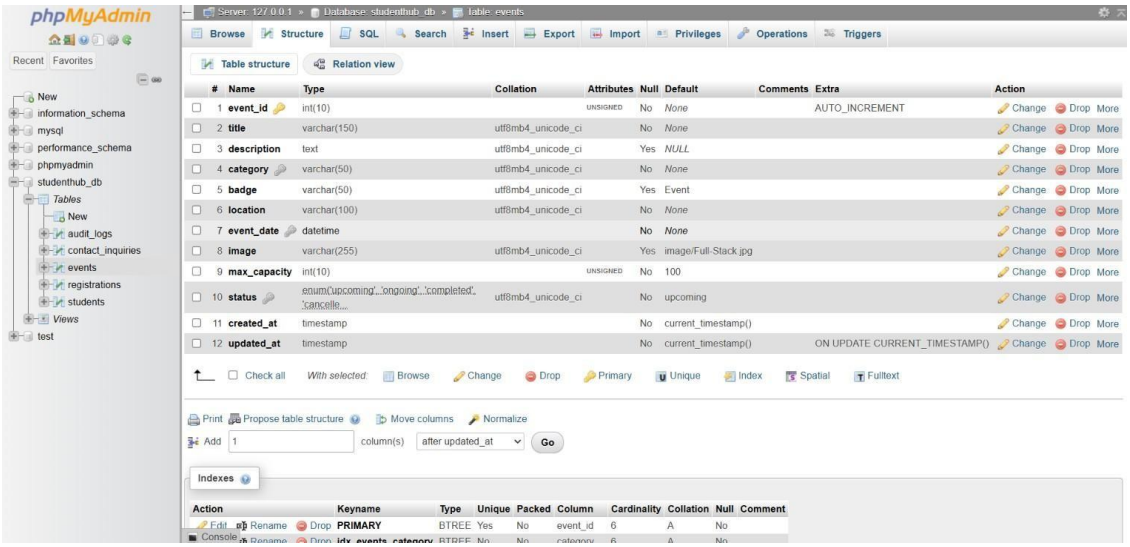
2.5 Output 5: Students Table — Seeded Records & Hashed Passwords

Data browsing view in phpMyAdmin displaying seeded student records with bcrypt-hashed passwords (prefixed with $2y$10$), unique student codes, academic departments, and auto-generated timestamps confirming successful INSERT operations via PDO prepared statements.



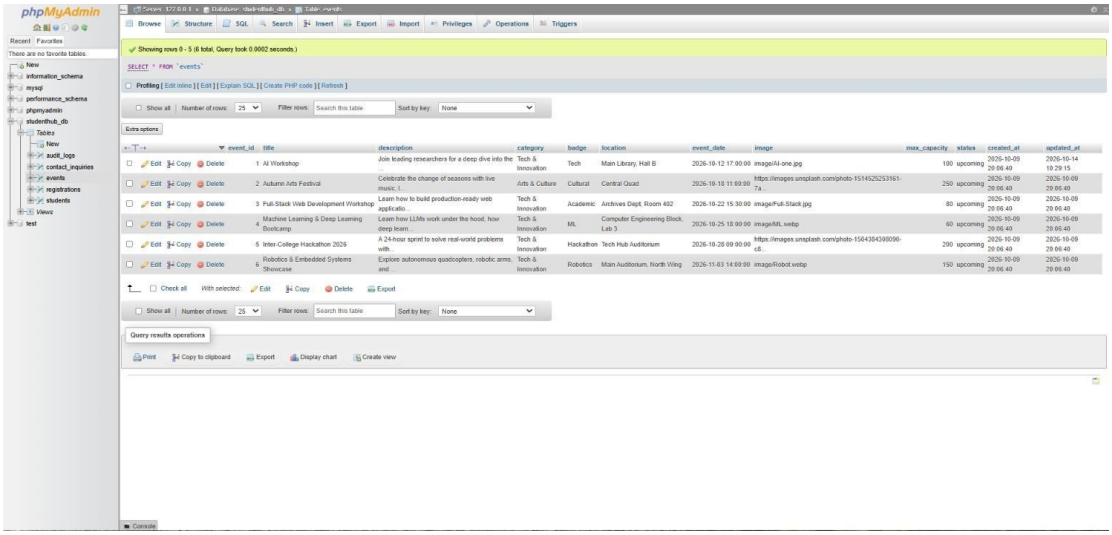
2.6 Output 6: Events Table Structure & Attribute Definitions

Table schema for the events table detailing primary key event_id, event title, rich description, category, badge, physical location, start/end dates, poster image path, max_capacity, status ENUM, and audit timestamps.



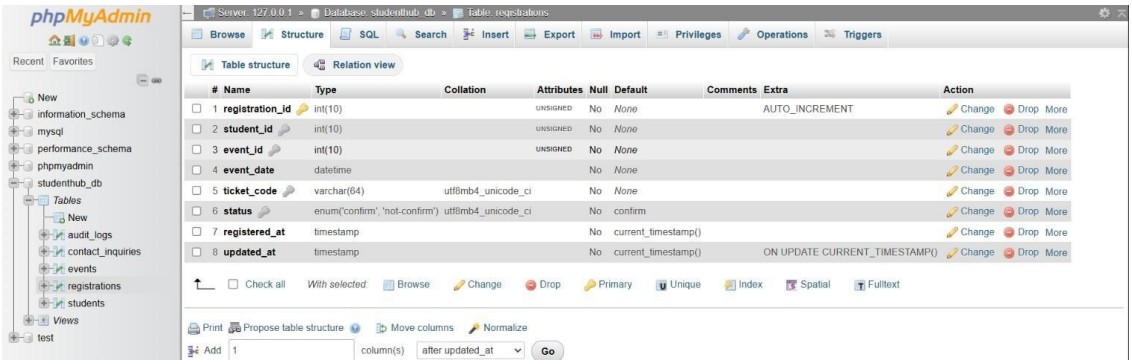
2.7 Output 7: Campus Events Catalog Seed Records (6 Events)

phpMyAdmin records view for events table containing 6 university events across categories (Tech & Innovation, Arts & Culture, Sports) with titles, descriptions, badge labels, venue locations, date ranges, capacities, and active status flags.



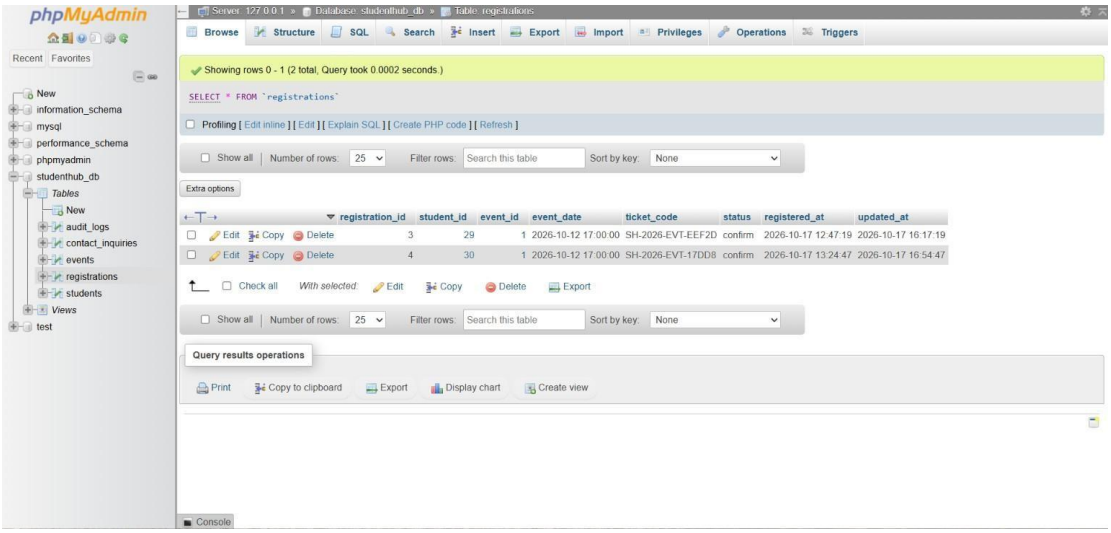
2.8 Output 8: Registrations Table Schema & Relational Constraints

Relational junction schema for registrations linking student_id and event_id with foreign key indexes, assigned ticket_code (VARCHAR 50, UNIQUE), status ENUM (confirmed/cancelled/pending), and created_at timestamp.



2.9 Output 9: Confirmed Event Registrations Dataset

Browsing view of verified student event registrations, showing active bookings with ticket reference codes SH-2026-EVT-EXXXX, confirmed status, and registration timestamps linking specific students to campus events through foreign key references.



3. Key Questions & Technical Analysis

Q1. Is the schema normalized properly?

Yes. The database design strictly adheres to Third Normal Form (3NF) principles across all operational tables:

• **First Normal Form (1NF):** All attributes store atomic, scalar values with no repeating groups or composite columns. Each field contains a single value of its declared type.

• **Second Normal Form (2NF):** The schema satisfies 1NF and all non-key attributes are fully functionally dependent on the entire primary key. No partial dependencies exist since each table uses a single-column surrogate primary key (student_id, event_id, registration_id).

• Third Normal Form (3NF): All transitive dependencies have been removed. Attributes such as department, email, and password depend exclusively on the primary key and not on any non-key attribute.

• **Relationship Decomposition:** The Many-to-Many relationship between students and events is decomposed into two clean One-to-Many relationships through the registrations junction table, eliminating data redundancy and update anomalies.

Q2. Are primary keys and foreign keys correctly defined?

Yes. Primary keys, foreign keys, unique constraints, and referential integrity rules are rigorously defined:

• **Primary Key Integrity:** Each table (students, events, registrations, contact_inquiries, audit_logs) employs an unsigned INT AUTO_INCREMENT primary key ensuring unique, non-null, monotonically increasing row identifiers.

• **Unique Key Constraints:** In the students table, email and roll_number carry separate UNIQUE constraints to prevent duplicate registrations at the database level.

• **Foreign Key Constraints:** The registrations table enforces foreign keys: fk_registrations_student (FOREIGN KEY (student_id) REFERENCES students(student_id)) and fk_registrations_event (FOREIGN KEY (event_id) REFERENCES events(event_id)).

• **Referential Action:** Foreign keys enforce relational integrity: modifying or deleting a referenced student or event automatically cascades to dependent registration records, preventing orphaned rows.

Q3. Is the database connection handled securely?

Yes. The database connection is handled securely through industry-standard PHP Data Objects (PDO) architecture:

• **Centralized Singleton / Secure Configuration (db.php):** Database credentials (host, dbname, user, password) are defined in an isolated backend file never exposed to the web root or client-side code.

• **Parameterized Prepared Statements:** All database queries utilize PDO prepared statements ($stmt = $pdo->prepare(...) and $stmt->execute([...])) to prevent SQL injection by separating query logic from data values.

• **Explicit Error Handling & Logging:** PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION is set in connection options. Database errors surface as catchable exceptions, enabling structured try-catch error handling rather than silent failures.

• **Emulated Prepares Disabled:** Setting PDO::ATTR_EMULATE_PREPARES => false forces MySQL's native prepared statement engine, ensuring type-safe parameterized execution at the database driver level.

• **Character Set Definition:** The connection string explicitly specifies charset=utf8mb4 to protect against character-encoding exploits and ensure full Unicode support across all operations.

4. GitHub Repository Link

GitHub Repository URL: https://github.com/smit-2406/studenthub-website

5. Conclusion & Learning Outcomes

Through the execution of Practical 8, relational database modeling, MySQL schema normalization, and secure PHP database connectivity were successfully implemented for the StudentHub university portal. A fully normalized schema with five tables (students, events, registrations, contact_inquiries, audit_logs) was deployed with proper primary keys, foreign key constraints, unique indexes, and ENUM-typed status fields. PDO-based connectivity with native prepared statements, exception-mode error handling, and utf8mb4 character encoding established a secure, injection-resistant data access layer. Seeded records verified INSERT operations, relational integrity, and bcrypt password hashing. This practical fulfills Course Outcomes CO1 and CO5 by establishing the persistent data architecture for all subsequent backend practicals.